

Semantic Search & Retrieval-Augmented Generation

Bi-Encoders · Cross-Encoders · Two-Stage Pipelines · RAG · Evaluation Metrics

► Sabancı University ► SentenceTransformers + HuggingFace ► Chapter 8 – Extended Notes

CONTENTS

1. Why Semantic Search?
2. Dense Retrieval — Bi-Encoders
 1. Architecture: Inside the Bi-Encoder
 2. Mean Pooling vs. [CLS] Pooling
 3. Similarity Functions
 4. Code Walkthrough (Step by Step)
 5. Limitations
3. Reranking — Cross-Encoders
 1. Architecture: Inside the Cross-Encoder
 2. Code Walkthrough
 3. Bi-Encoder vs Cross-Encoder
4. Training Bi- and Cross-Encoders
5. Two-Stage Pipeline
6. Chunking Long Texts
7. Retrieval-Augmented Generation
 1. Full RAG Code
 2. Advanced RAG Patterns

8. Evaluation Metrics

1. MAP — Worked Example
2. nDCG — Worked Example
3. RAG Evaluation (Ragas)

9. Summary & Quick-Start Models

01 Why Semantic Search?

The problem with keyword search, and how language models change everything.

Traditional search engines — and most database LIKE queries — work by matching **exact words**. This is called *lexical search* or *keyword search*. The most popular algorithm is **BM25** (Best Match 25), a refinement of TF-IDF.

Consider this example from a corpus about the film *Interstellar*:

- ▶ **Query:** "how precise was the science"
- ▶ **Best matching document:** "It has also received praise from many astronomers for its scientific accuracy and portrayal of theoretical astrophysics."

The query contains the words *precise* and *science*. The best document contains *scientific* and *accuracy* — **none of the query's keywords appear verbatim**. BM25 completely misses this document. It instead returns a document that contains the word "science" literally: "*Interstellar is a 2014 epic science fiction film.*" — which does not answer the question at all.



The Key Insight

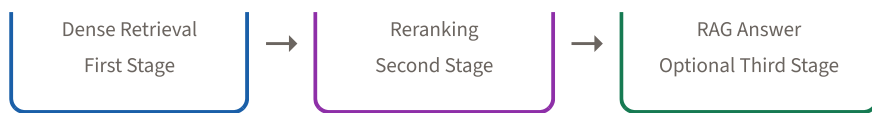
Semantic search does not match words — it matches *meaning*. The words "precise" and "accuracy" are semantically close even though they are lexically different. To achieve this, we need to represent the meaning of text as a **vector in a continuous space** (an embedding), where texts with similar meanings are geometrically close to each other.

There are three major approaches using language models for search, typically combined into a pipeline:

Bi-Encoder

Cross-Encoder

LLM Generator



The rest of this document explains each stage in depth.

02 Dense Retrieval — Bi-Encoders

How text is compressed into a single vector, and how we search with it at scale.

A **bi-encoder** is a neural network that converts any piece of text into a dense vector (a list of floating-point numbers). Once we have these vectors, searching becomes a matter of finding the closest vectors to the query vector in the embedding space — a task called *nearest-neighbor search*.

2.1 Architecture: Inside the Bi-Encoder

The model at the heart of a bi-encoder is typically a pre-trained transformer like BERT or RoBERTa. Here is exactly what happens, step by step:

1. **Tokenization.** The input text is split into subword tokens. Special tokens [CLS] (start) and [SEP] (end) are added automatically.
2. **Embedding lookup.** Each token gets an initial vector from a learned embedding table (like Word2Vec but trainable).
3. **Transformer layers.** The tokens pass through multiple layers of *self-attention + feed-forward* networks. After these layers, each token has a *contextual* vector that encodes not just the token's identity, but also its relationship to every other token in the sequence. So the word "bank" will have different vectors in "river bank" vs "bank account."
4. **Pooling.** We have one vector per token (e.g., 512 tokens × 768 dimensions). We need to collapse this into a **single vector** representing the whole sentence. The most common approach is [mean pooling](#).
5. **Optional normalization.** The final vector is often L2-normalized (divided by its own length) so all vectors lie on a unit hypersphere, making cosine similarity equivalent to dot product.

FIGURE — BI-ENCODER FORWARD PASS (DETAILED)

QUERY BRANCH — encoded independently

INPUT QUERY

"How precise was the science?"

AFTER TOKENIZATION

[CLS] How precise was the science ? [SEP]



BERT / RoBERTa Encoder

Multi-head self-attention over all tokens · 12 layers · 768 hidden dims



[CLS] How precise was the science ? [SEP]
h₀ h₁ h₂ h₃ h₄ h₅ h₆ h₇

Each token has its own 768-dimensional contextual vector. The word "science" knows it's in a question context.

↓ Mean Pooling

FINAL QUERY EMBEDDING (768-DIM)

$q = \text{mean}(h_0, h_1, h_2, \dots, h_7) = [0.21, -0.43, 0.87, 0.12, \dots]$

DOCUMENT BRANCH — encoded independently (can be done offline!)

INPUT DOCUMENT

"...praised by astronomers for its scientific accuracy..."

AFTER TOKENIZATION

[CLS] praised by astronom ##ers for its scientific accuracy ... [SEP]

↓ Same Encoder → Mean Pool

FINAL DOCUMENT EMBEDDING (768-DIM)

$d = [0.19, -0.41, 0.85, 0.11, \dots]$

SIMILARITY COMPUTATION

$$\cos(q, d) = (q \cdot d) / (\|q\| \cdot \|d\|) = 0.89$$

The two vectors are geometrically close because "precision" and "scientific accuracy" have similar meaning. **Neither encoder saw the other's text.**

2.2 Mean Pooling vs [CLS] Pooling — The Critical Design Choice

After the encoder, we have one hidden state vector per token. We need exactly **one vector for the whole sentence**. There are two main approaches:

APPROACH	HOW IT WORKS	STRENGTHS	WEAKNESSES
[CLS] Token	Take only the hidden state of the [CLS] token. BERT was designed so this token attends to the whole sequence.	Simple; what BERT was pre-trained to do for classification	Doesn't work well out-of-the-box for semantic similarity without fine-tuning. [CLS] may not capture sentence meaning well.
Mean Pooling	Average the hidden states of ALL tokens (excluding padding). Every token contributes equally to the final embedding.	Captures more of the sentence; generally outperforms [CLS] for sentence embeddings; default in SentenceTransformers	Stop words (the, a, is) may dilute the signal — sometimes mitigated by weighted pooling
Max Pooling	Take the maximum value across all token positions for each dimension.	Highlights the most prominent features	Rarely used in practice for retrieval



The Sentence-BERT Paper (2019) — Why This Matters

Reimers & Gurevych showed that vanilla BERT with [CLS] pooling performed *worse than averaging GloVe embeddings* on sentence similarity tasks. They introduced **SBERT (Sentence-BERT)**: fine-tune BERT with a siamese architecture and mean pooling using a contrastive loss. This is the foundation of the `sentence-transformers` library you use today.

2.3 Similarity Functions

Once we have two embedding vectors, we need a scalar similarity score. The two main options are:

$$\text{cosine similarity} = (\mathbf{u} \cdot \mathbf{v}) / (\|\mathbf{u}\| \times \|\mathbf{v}\|) \in [-1, 1]$$

Dot product divided by the product of magnitudes. Measures the *angle* between vectors, not their magnitude.

Standard for semantic search.

$$\text{dot product} = \mathbf{u} \cdot \mathbf{v} = \sum \mathbf{u}_i \times \mathbf{v}_i \in (-\infty, +\infty)$$

Faster to compute. If vectors are L2-normalized first (divided by their length), dot product = cosine similarity. **Use this with FAISS for speed.**

2.4 Code Walkthrough — Step by Step

We will build a complete semantic search system from scratch. The corpus is the Interstellar Wikipedia passage from the textbook. Each step is annotated carefully.

Step 1 – Install libraries

BASH

```
# Install once
pip install sentence-transformers faiss-cpu
```

Step 2 – Load model and prepare corpus

PYTHON

```
from sentence_transformers import SentenceTransformer
import numpy as np
import faiss

# Load a pre-trained bi-encoder model.
# 'multi-qa-mpnet-base-dot-v1' is specifically trained on Q&A pairs
# so it understands the asymmetry between a question and its answer.
model = SentenceTransformer('multi-qa-mpnet-base-dot-v1')

# Our document corpus – each string is one "chunk" (one sentence here).
corpus = [
    "Interstellar is a 2014 epic science fiction film directed by Christopher Nolan",
    "The film stars Matthew McConaughey, Anne Hathaway, Jessica Chastain, and Micha",
    "Set in a dystopian future, astronauts travel through a wormhole near Saturn.",
    "It received praise from astronomers for its scientific accuracy and portrayal",
    "Kip Thorne, a 2017 Nobel laureate in Physics, acted as a scientific consultant",
    "Interstellar premiered on October 26, 2014, in Los Angeles.",
    "The film had a worldwide gross over $677 million, and $773 million with subseq",
    "It was nominated for five awards at the 87th Academy Awards, winning Best Visu",
]
```

Step 3 – Embed the corpus (the expensive step – done ONCE, stored offline)

PYTHON

```
# encode() runs each sentence through the transformer + mean pooling.
# Input: list of strings
# Output: numpy array of shape (num_sentences, embedding_dim)
```

```

#
# 'batch_size=32' means 32 sentences are processed simultaneously (faster).
# 'show_progress_bar=True' shows a tqdm bar – useful for large corpora.

doc_embeddings = model.encode(
    corpus,
    batch_size=32,
    show_progress_bar=True,
    convert_to_numpy=True
)

print(f"Shape: {doc_embeddings.shape}")
# → Shape: (8, 768)
# 8 sentences, each represented as 768 floats

```

Step 4 – Build the FAISS index (for fast nearest-neighbor search)

PYTHON

```

# Why FAISS?
# If we have 1 million documents, computing cosine similarity against ALL of
# them for every query would take seconds. FAISS uses approximate nearest-
# neighbor (ANN) algorithms (like HNSW or IVF) to answer in milliseconds.
#
# Here we normalize vectors first so that dot product = cosine similarity,
# then use IndexFlatIP (Inner Product = dot product).

# L2-normalize so all vectors have length 1
doc_emb_norm = doc_embeddings.copy()
faiss.normalize_L2(doc_emb_norm)

dim = doc_emb_norm.shape[1]      # 768
index = faiss.IndexFlatIP(dim)   # Inner Product index
index.add(doc_emb_norm)          # Add all doc vectors

print(f"Index contains {index.ntotal} vectors.")
# → Index contains 8 vectors.

```

Step 5 – Search function

PYTHON

```

def semantic_search(query: str, top_k: int = 3):
    """
    1. Embed the query with the SAME model (a separate forward pass).
       The query sees ONLY its own tokens – it never sees the documents.
    2. Normalize the query embedding.
    3. Ask FAISS for the top_k most similar document vectors.
    4. Return the results sorted by similarity score (higher = more similar).
    """
    q_emb = model.encode([query], convert_to_numpy=True)

```

```

faiss.normalize_L2(q_emb)

# index.search returns (distances, indices), shape: (1, top_k)
scores, idxs = index.search(q_emb, top_k)

print(f"\n🔍 Query: '{query}')"
print("-" * 65)
for rank, (score, idx) in enumerate(zip(scores[0], idxs[0]), 1):
    print(f"  [{rank}] score={score:.4f} → {corpus[idx]}")

semantic_search("how precise was the science")
semantic_search("box office earnings")
semantic_search("when did the film premiere")

```

OUTPUT

🔍 Query: 'how precise was the science'

```

[1] score=0.7231 It received praise from astronomers for its scientific accuracy...
[2] score=0.6104 Rip Thorne, a 2017 Nobel laureate in Physics, acted as a scientific
consultant.
[3] score=0.4810 Interstellar is a 2014 epic science fiction film directed by
Christopher Nolan.

```

🔍 Query: 'box office earnings'

```

[1] score=0.7589 The film had a worldwide gross over $677 million...

```



What just happened?

The query "how precise was the science" contains neither "praised", "astronomers", nor "accuracy." Yet the bi-encoder correctly identifies that sentence as the most relevant — because *both vectors point in a similar direction in embedding space*. The model learned during training that "precise science" and "scientific accuracy" are semantically equivalent concepts.

2.5 Limitations of Bi-Encoders

- ▶ **Compression bottleneck.** All meaning must be compressed into a single vector (e.g., 768 floats). For long or complex documents, important details can be "averaged away." This is called the *representation bottleneck*.
- ▶ **No query-document interaction during encoding.** The model never sees both texts simultaneously, so it cannot model subtle interactions like negation ("NOT expensive") or comparative relationships.

- ▶ **Query–document style asymmetry.** A question ("When was X released?") is linguistically very different from its answer ("X was released in 2014"). Models must be explicitly trained on question–answer pairs to bridge this gap.
- ▶ **Out-of-domain degradation.** A model trained on Wikipedia performs poorly on legal or medical texts unless fine-tuned on domain data.
- ▶ **Exact-match queries.** For queries like "find the exact phrase 'neural manifold hypothesis'", keyword search is better. Hybrid search (BM25 + bi-encoder) is recommended for production.

03 Reranking — Cross-Encoders

When accuracy matters more than speed: letting the model see both texts at once.

A **cross-encoder** takes a query and a document as a *single combined input* and produces a **single scalar relevance score**. Unlike the bi-encoder, the model sees both texts simultaneously and can model all interactions between every token of the query and every token of the document.

3.1 Architecture: Inside the Cross-Encoder

FIGURE — CROSS-ENCODER FORWARD PASS (DETAILED)

INPUT — Query and Document are CONCATENATED into a single sequence:



■ Purple tokens = query ■ Red tokens = document ■ Grey = special separators

SELF-ATTENTION — Every token attends to every other token (including across the [SEP]):

Attention heat map (darker = stronger attention):

	[CLS]	prec-	ise	[SEP]	scien-	tific	accur-	acy
precise →								
accuracy →								

The word "precise" (query) strongly attends to "accurate" and "accuracy" (document) — capturing the semantic relationship the bi-encoder approximates implicitly.

BERT / RoBERTa Encoder — Full self-attention across ALL tokens (query + document combined)

[CLS] CONTEXTUAL VECTOR — ENCODES THE RELATIONSHIP BETWEEN QUERY AND DOCUMENT

$h_{[CLS]} \rightarrow \text{Linear}(768 \rightarrow 1) \rightarrow \text{Sigmoid} \rightarrow \text{relevance} \in [0, 1]$

Final Relevance Score

0.94

Very high relevance — the model confirms the document answers the query.



Why Cross-Encoders Cannot Replace Bi-Encoders

For every query, the cross-encoder must process *each candidate document separately*: one forward pass per (query, document) pair. With 1 million documents and a BERT-base model at ~20ms per pass: **$1,000,000 \times 20\text{ms} = 5.5 \text{ hours per query}$** . This is completely impractical. The solution: use a bi-encoder to shortlist 100–500 candidates in milliseconds, then apply the cross-encoder only to that shortlist.

3.2 Code Walkthrough — Cross-Encoder Reranking

Reranking with CrossEncoder — step by step

PYTHON

```
from sentence_transformers import CrossEncoder

# Load a cross-encoder trained on MS MARCO (8.8M query-passage pairs).
# This model outputs raw logits (not bounded to [0,1]).
# Higher logit = more relevant.
reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

query = "how precise was the science"

# Imagine these are the top-10 results from the bi-encoder Stage 1.
# The bi-encoder found them quickly but may not have ordered them perfectly.
stage1_results = [
    "Interstellar is a 2014 epic science fiction film directed by Christopher Nolan",
    "The film stars Matthew McConaughey, Anne Hathaway, Jessica Chastain.",
    "Set in a dystopian future, astronauts travel through a wormhole near Saturn.",
    "It received praise from astronomers for its scientific accuracy.",
    "Kip Thorne, a 2017 Nobel laureate, acted as a scientific consultant.",
    "The film had a worldwide gross over $677 million.",
    "Interstellar premiered on October 26, 2014, in Los Angeles.",
    "It was nominated for five awards at the 87th Academy Awards.",
]
```

```
# Build (query, document) pairs – the cross-encoder needs BOTH together.
# This is the key difference from bi-encoder: the model sees both at once.
pairs = [[query, doc] for doc in stage1_results]

# predict() runs one forward pass per pair and returns a score for each.
# Shape: (num_pairs,) – one scalar per (query, doc) pair.
scores = reranker.predict(pairs)

# Sort pairs by score (descending)
ranked = sorted(zip(scores, stage1_results), key=lambda x: x[0], reverse=True)

print(f"Re-ranked results for: '{query}'\n")
for rank, (score, doc) in enumerate(ranked, 1):
    print(f" [{rank}] score={score:+.3f} {doc[:70]}")
```

OUTPUT – CROSS-ENCODER RERANKING

```
[1] +8.721    It received praise from astronomers for its scientific accuracy.
[2] +5.103    Kip Thorne, a 2017 Nobel laureate, acted as a scientific consultant.
[3] -1.344    Interstellar is a 2014 epic science fiction film directed by
Christopher Nolan.
[4] -4.201    The film had a worldwide gross over $677 million.
[5] -6.883    It was nominated for five awards at the 87th Academy Awards.
```

Note: scores are raw logits – negative does NOT mean irrelevant in absolute terms, only that other results are more relevant for this query. Apply sigmoid() for probabilities if needed.

3.3 Bi-Encoder vs Cross-Encoder — Complete Comparison

PROPERTY	BI-ENCODER	CROSS-ENCODER
Input format	Query and document encoded separately , independently	Query + document concatenated into one single sequence
Attention	Self-attention <i>within</i> each text only — query never sees document tokens	Full self-attention <i>across both</i> texts — every query token attends to every document token
Output	Two embedding vectors → cosine/dot similarity score	Single scalar relevance score directly
Document pre-computation	✅ Yes — embed all docs once offline, store in vector DB	❌ No — must process query+doc together, cannot pre-compute

PROPERTY	BI-ENCODER	CROSS-ENCODER
Inference speed	Very fast — ANN search, <10ms for millions of docs	Slow — one forward pass per (query, doc) pair, $O(n)$ with corpus size
Accuracy	Good — limited by embedding compression bottleneck	Excellent — full cross-attention captures subtle relationships
Typical corpus size	Millions to billions of documents	Dozens to a few hundred candidates (after first stage)
Role in pipeline	Stage 1 — maximize recall: retrieve broadly	Stage 2 — maximize precision: reorder shortlist
Training input	Triples: (anchor, positive_doc, negative_doc)	Pairs with labels: (query, doc, relevance_score)
Training loss	MultipleNegativesRankingLoss, TripletLoss, CosineSimilarityLoss	BinaryCrossEntropyLoss, CrossEntropyLoss (for graded relevance)
Library class	SentenceTransformer('model-name')	CrossEncoder('model-name')

04

Training Bi- and Cross-Encoders

Dataset construction, loss functions, and how to fine-tune on your own data.

Both architectures start from a pre-trained BERT/RobERTa base and are *fine-tuned* with different objectives. The key differences are in **(1) how the dataset is constructed** and **(2) which loss function is used**.

4.1 Bi-Encoder: Dataset & MultipleNegativesRankingLoss

DATASET CONSTRUCTION

You need **(query, positive_document)** pairs — that's all. No explicit negatives required. Common sources:

- **Natural Questions / TriviaQA:** Wikipedia questions + the paragraph containing the answer

- ▶ **MS MARCO (passage retrieval split):** Bing queries + clicked passage
- ▶ **Your own data:** FAQ pairs, support tickets + resolved answers, search logs with click-through

Training examples (bi-encoder)

PAIR 1

Q: "When was Interstellar released?"

D*: "Premiered Oct 26, 2014 in Los Angeles."

PAIR 2

Q: "Who directed the film?"

D*: "Directed by Christopher Nolan."

PAIR 3

Q: "How much did it gross?"

D*: "Worldwide gross over \$677 million."

PAIR 4

Q: "Who starred in it?"

D*: "Stars Matthew McConaughey and Anne Hathaway."

These 4 pairs form one batch (batch_size=4). During loss computation, **D* of each other pair automatically becomes a negative.**

What the model "sees" for Q1

For query Q1 = "When was Interstellar released?", the loss is computed against all 4 documents in the batch:

POSITIVE (LABEL = CORRECT)

D1: "Premiered Oct 26, 2014 in Los Angeles."

IN-BATCH NEGATIVE (AUTOMATIC)

D2: "Directed by Christopher Nolan."

IN-BATCH NEGATIVE (AUTOMATIC)

D3: "Worldwide gross over \$677 million."

IN-BATCH NEGATIVE (AUTOMATIC)

D4: "Stars Matthew McConaughey and Anne Hathaway."

With batch_size=32 → 31 negatives per query. Larger batch = harder training = better model.

MULTIPLENEGATIVESRANKINGLOSS – STEP BY STEP

The loss function turns this into a classification problem over the batch: *"which document is the correct one for this query?"*

STEP-BY-STEP CALCULATION (BATCH_SIZE = 4)

Step 1 – Compute similarity scores for Q1 against all 4 documents (dot product after normalization):

```
sim(Q1, D1) = 0.89 ← correct pair
sim(Q1, D2) = 0.31
sim(Q1, D3) = 0.24
sim(Q1, D4) = 0.19
```

Step 2 – Apply softmax over these 4 scores (this is exactly softmax, same as in cross-entropy classification):

```
softmax score for D1 = exp(0.89) /
[exp(0.89)+exp(0.31)+exp(0.24)+exp(0.19)]
= 2.435 / [2.435+1.363+1.271+1.209]
= 2.435 / 6.278 = 0.388
```

Step 3 – Compute loss (cross-entropy: penalize for not giving D1 a probability of 1.0):

```
loss(Q1) = -log(0.388) = 0.946
```

After training converges, the model pushes sim(Q1,D1) much higher than the rest:

```
sim(Q1, D1) = 0.97 ← very high
sim(Q1, D2) = 0.08
sim(Q1, D3) = 0.05
sim(Q1, D4) = 0.03
```

```
loss(Q1) = -log(exp(0.97)/[exp(0.97)+...]) ≈ -log(0.98) ≈ 0.020 ← near
zero ✓
```

```
Total batch loss = mean of [loss(Q1) + loss(Q2) + loss(Q3) +
loss(Q4)]
```

Backpropagation updates encoder weights to minimize this loss.



False Negatives — The One Caveat

In-batch negatives assume that D2, D3, D4 are truly irrelevant to Q1. But what if D3 ("gross over \$677 million") happens to also partially answer Q1? This is called a **false negative** — a relevant document treated as negative. It can slightly hurt training. Mitigation: deduplicate your corpus, or use *hard negative mining* (run bi-encoder first, find near-miss docs, label them properly).

Bi-encoder fine-tuning with MultipleNegativesRankingLoss

PYTHON

```
from sentence_transformers import SentenceTransformer, InputExample, losses
from torch.utils.data import DataLoader

model = SentenceTransformer('multi-qa-mpnet-base-dot-v1')

# Only (query, positive_doc) pairs needed – no explicit negatives!
# In-batch negatives are handled automatically by the loss function.
train_examples = [
    InputExample(texts=['When was Interstellar released?',
                        'Premiered Oct 26, 2014 in Los Angeles.']),
    InputExample(texts=['Who directed the film?',
                        'Directed by Christopher Nolan.']),
    InputExample(texts=['How much did it gross?',
                        'Worldwide gross over $677 million.']),
    InputExample(texts=['Who starred in it?',
                        'Stars Matthew McConaughey and Anne Hathaway.']),
    # ... add hundreds or thousands more pairs
]

# IMPORTANT: larger batch_size = more in-batch negatives = better training
# Try 16, 32, or 64 depending on your GPU memory
train_loader = DataLoader(train_examples, shuffle=True, batch_size=32)

# The loss internally:
# 1. Encodes all queries and documents in the batch
# 2. Computes a (batch_size × batch_size) similarity matrix
# 3. Applies cross-entropy: diagonal should be high, off-diagonal low
loss_fn = losses.MultipleNegativesRankingLoss(model)

model.fit(
    train_objectives=[(train_loader, loss_fn)],
    epochs=3,
    warmup_steps=100,
```

```
    output_path='./my_retriever'  
)
```

4.2 Cross-Encoder: Dataset & Binary Cross-Entropy Loss

DATASET CONSTRUCTION

Cross-encoder training needs **(query, document, relevance_label)** triples. The label can be binary (0/1) or graded (0/1/2/3). The key challenge is getting *hard negatives* — documents that look relevant but aren't.



MS MARCO — The Standard Dataset (8.8M pairs)

Microsoft collected this from Bing search logs. A user types a query → Bing returns passages → user **clicks one** → that passage gets `label=1` . **Passages that were shown but not clicked** get `label=0` . This is called *implicit relevance feedback*. The "not clicked" passages are often hard negatives — they were returned by Bing because they seem related, but aren't the right answer.

Easy negatives (not useful)

POSITIVE — LABEL=1

Q: "What is the capital of Australia?"

D: "Canberra is the capital city of Australia."

EASY NEGATIVE — LABEL=0

Q: "What is the capital of Australia?"

D: "Photosynthesis converts sunlight into glucose..."

The model instantly learns to score this 0. No real learning happens. Training on only easy negatives produces a weak model.

Hard negatives (necessary)

POSITIVE — LABEL=1

Q: "What is the capital of Australia?"

D: "Canberra is the capital city of Australia."

HARD NEGATIVE — LABEL=0

Q: "What is the capital of Australia?"

D: "Sydney is the largest city in Australia and a major hub."

The model must learn the subtle distinction: "largest city" $\neq$ "capital." This is hard — it forces genuine learning of relevance.

HOW TO GET HARD NEGATIVES FOR YOUR OWN DATASET

1. **BM25 negatives:** Run BM25 on your corpus, take top-k results that are NOT relevant. These share keywords but may not answer the question.
2. **Bi-encoder negatives (best practice):** Run a trained bi-encoder, take its top-k results that are NOT relevant. These are the hardest — the bi-encoder already thinks they're relevant.
3. **Random negatives:** Pick random documents from the corpus. Easy, but weak.

BINARY CROSS-ENTROPY LOSS — STEP BY STEP

The cross-encoder outputs a raw logit (any real number). We apply sigmoid to get a probability, then compute BCE loss.

$$p = \text{sigmoid}(\text{raw_score}) = 1 / (1 + e^{-\text{score}})$$

Squashes any real number into (0, 1). A raw score of 0 $\rightarrow$ $p=0.5$. Score of +3 $\rightarrow$ $p=0.95$. Score of -3 $\rightarrow$ $p=0.047$.

$$\text{BCE loss} = - [y \cdot \log(p) + (1-y) \cdot \log(1-p)]$$

$y \in \{0, 1\}$ is the true label. p is the model's predicted probability. When $y=1$, only $\log(p)$ matters. When $y=0$, only $\log(1-p)$ matters.

FOUR CONCRETE SCENARIOS

✓ CORRECT: POSITIVE PAIR, HIGH SCORE

$y = 1$ (relevant)
 $\text{raw} = +2.2$
 $p = \text{sigmoid}(2.2) = 0.90$
 $\text{loss} = -\log(0.90) = 0.105$ ✓ low

✗ WRONG: POSITIVE PAIR, LOW SCORE

$y = 1$ (relevant)
 $\text{raw} = -1.5$
 $p = \text{sigmoid}(-1.5) = 0.18$
 $\text{loss} = -\log(0.18) = 1.715$ ✗ high

✓ CORRECT: NEGATIVE PAIR, LOW SCORE

$y = 0$ (not relevant)
 $\text{raw} = -2.8$
 $p = \text{sigmoid}(-2.8) = 0.057$

✗ WRONG: NEGATIVE PAIR, HIGH SCORE

$y = 0$ (not relevant)
 $\text{raw} = +1.8$
 $p = \text{sigmoid}(1.8) = 0.858$

loss = $-\log(1-0.057) = -\log(0.943) =$
0.059 ✓

loss = $-\log(1-0.858) = -\log(0.142) =$
1.952 ✗

The formula's elegance: when $y=1$, the $(1-y) \cdot \log(1-p)$ term vanishes → only $\log(p)$ matters → "make p high".

When $y=0$, the $y \cdot \log(p)$ term vanishes → only $\log(1-p)$ matters → "make p low".

Backprop adjusts weights to reduce loss across all pairs in the batch.

Cross-encoder fine-tuning with Binary Cross-Entropy

PYTHON

```
from sentence_transformers import CrossEncoder, InputExample
from torch.utils.data import DataLoader

# Start from a pre-trained cross-encoder
reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2', num_labels=1)

# Training data: (query, document, label) triples
# label=1 → relevant, label=0 → not relevant
# Mix of easy and hard negatives for best results
train_examples = [
    # Positive pairs (label=1)
    InputExample(texts=['When was Interstellar released?',
                        'Premiered Oct 26, 2014 in Los Angeles.'], label=1),
    # Hard negatives (label=0) – returned by bi-encoder but wrong
    InputExample(texts=['When was Interstellar released?',
                        'Interstellar is a 2014 science fiction film.'], label=0),
    InputExample(texts=['Who directed the film?',
                        'Directed by Christopher Nolan.'], label=1),
    InputExample(texts=['Who directed the film?',
                        'Nolan produced and co-wrote the screenplay.'], label=0),
    # ... many more examples
]

train_loader = DataLoader(train_examples, shuffle=True, batch_size=16)

# fit() automatically uses BCEWithLogitsLoss for num_labels=1
# The model outputs one logit → sigmoid → probability → BCE loss
reranker.fit(
    train_data_loader=train_loader,
    epochs=3,
    warmup_steps=100,
    output_path='./my_reranker'
)
```

The Two-Stage Pipeline

Combining the speed of bi-encoders with the precision of cross-encoders.

The bi-encoder and cross-encoder are designed to complement each other. In production, you never pick just one:

Stage 1 Bi-Encoder

Retrieve top-K candidates

Embed query → ANN search over millions of docs → return top 100. Goal: [maximize recall](#) . If a relevant doc is not in this list, it can never be recovered.

~10ms

for 1M docs

Stage 2 Cross-Encoder

Rerank the top-K candidates

Run (query, doc) through cross-encoder for each of the ~100 candidates. Goal: [maximize precision](#) . Correct ordering within the shortlist.

~200ms

for 100 candidates

Output

Top-5 or Top-10 final results

Return to user, or pass to an LLM for grounded generation (RAG).

<300ms

total

```

from sentence_transformers import SentenceTransformer, CrossEncoder, util

bi_encoder = SentenceTransformer('multi-qa-mpnet-base-dot-v1')
cross_encoder = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')

# Pre-compute and store – run this ONCE for your corpus
corpus_embeddings = bi_encoder.encode(corpus, convert_to_tensor=True, show_progress

def retrieve_and_rerank(query: str, stage1_k=25, stage2_k=5):
    # — STAGE 1: Bi-encoder retrieval —————
    q_emb = bi_encoder.encode(query, convert_to_tensor=True)
    # semantic_search returns list of dicts: [{'corpus_id': int, 'score': float}, .
    hits = util.semantic_search(q_emb, corpus_embeddings, top_k=stage1_k)[0]

    print(f"Stage 1 (Bi-Encoder) top {stage1_k} results:")
    for h in hits[:3]:
        print(f"  BE score={h['score']:.3f} | {corpus[h['corpus_id']][:60]}")

    # — STAGE 2: Cross-encoder reranking —————
    # Build (query, candidate_doc) pairs for every Stage 1 hit
    ce_pairs = [[query, corpus[h['corpus_id']]] for h in hits]
    ce_scores = cross_encoder.predict(ce_pairs)

    # Attach CE scores back to hits
    for h, s in zip(hits, ce_scores):
        h['ce_score'] = float(s)

    # Re-sort by CE score (this may change the order significantly)
    reranked = sorted(hits, key=lambda x: x['ce_score'], reverse=True)

    print(f"\nStage 2 (Cross-Encoder) reranked top {stage2_k}:")
    for i, h in enumerate(reranked[:stage2_k], 1):
        print(f"  [{i}] CE={h['ce_score']:.2f} BE={h['score']:.3f} | {corpus[h['co

    return reranked[:stage2_k]

retrieve_and_rerank("how precise was the science")

```



Real-World Impact

On the multilingual MIRACL benchmark: bi-encoder alone achieves **nDCG@10 = 36.5**. Adding a cross-encoder reranker boosts this to **nDCG@10 = 62.8** — a 72% relative improvement in search quality, for <200ms of extra latency.

06 Chunking Long Texts

Transformer encoders have a token limit. Here is how to handle long documents correctly.

BERT-based models typically support a maximum of **512 tokens** (roughly 380 words). Most real documents — research papers, legal contracts, web pages — are much longer. We must split them into *chunks* before embedding.

The choice of chunking strategy has a direct effect on retrieval quality. There are three common approaches:

① Sentence-Level

S1: "Interstellar is a 2014 film..."

S2: "It stars Matthew McConaughey..."

S3: "Set in a dystopian future..."

- ✓ Fine-grained, precise matches
- ✗ Each sentence may lack context. "It" in S2 needs S1 to make sense.

② Paragraph / Fixed Window

Chunk 1: S1 + S2 + S3 + S4
(every 3–8 sentences)

Chunk 2: S5 + S6 + S7 + S8

- ✓ Good context within each chunk
- ✗ Information spanning the chunk boundary may be missed

③ Overlapping Chunks ★

Chunk 1: S1+S2+S3+S4

Overlap: S3+S4

Chunk 2: S3+S4+S5+S6

- ✓ No information is lost at boundaries
- ✓ Recommended for most production systems
- ✗ More chunks → larger index



Practical Rule of Thumb

Use **paragraph-level chunks of 3–8 sentences with 1–2 sentence overlap**, and **prepend the document title** to each chunk for context: "Interstellar (Film): It stars Matthew McConaughey..." . This gives each chunk self-contained meaning while preserving boundary context.

07

Retrieval-Augmented Generation (RAG)

Grounding an LLM's answers in retrieved facts to reduce hallucinations.

LLMs like GPT or Llama generate fluent text but *cannot reliably access specific facts*. Their knowledge is frozen at training time, and they sometimes "hallucinate" — fabricating plausible-sounding but incorrect answers. **RAG** solves this by dynamically retrieving relevant documents at inference time and injecting them into the LLM's prompt as context.



Key Paper: Lewis et al., 2020

"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" introduced RAG as a framework: a pre-trained retriever (bi-encoder) combined with a sequence-to-sequence generator, with gradients flowing through both. Modern RAG systems use this same idea but with frozen retrievers and generators connected only through the prompt.

7.1 Full RAG Code — HuggingFace + FAISS + Flan-T5

Complete RAG pipeline — retrieval + generation

PYTHON

```
from sentence_transformers import SentenceTransformer, CrossEncoder, util
from transformers import pipeline
import torch
```

```
# — 1. Retriever setup —————
bi_encoder = SentenceTransformer('multi-qa-mpnet-base-dot-v1')
cross_encoder = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
corpus_embeddings = bi_encoder.encode(corpus, convert_to_tensor=True)
```

```
# — 2. Generator setup —————
# Flan-T5 is a good free model for instruction-following and Q&A.
# For better quality, substitute: 'microsoft/Phi-3-mini-4k-instruct'
```

```
# or 'Qwen/Qwen2.5-1.5B-Instruct'
```

```
generator = pipeline(  
    "text2text-generation",  
    model="google/flan-t5-large",  
    device=0 if torch.cuda.is_available() else -1,  
)
```

```
# — 3. RAG function —————
```

```
def rag_answer(question: str, top_retrieve=10, top_context=3):  
    print(f"\n{' '*65}")  
    print(f"Question: {question}")  
    print(f"\n{' '*65}")  
  
    # STEP 1 – Retrieve with bi-encoder  
    q_emb = bi_encoder.encode(question, convert_to_tensor=True)  
    hits = util.semantic_search(q_emb, corpus_embeddings, top_k=top_retrieve)[0]  
  
    # STEP 2 – Rerank with cross-encoder  
    ce_pairs = [[question, corpus[h['corpus_id']]] for h in hits]  
    ce_scores = cross_encoder.predict(ce_pairs)  
    for h, s in zip(hits, ce_scores): h['ce_score'] = float(s)  
    reranked = sorted(hits, key=lambda x: x['ce_score'], reverse=True)[:top_context]  
  
    print(f"\n📖 Retrieved context passages:")  
    context_parts = []  
    for i, h in enumerate(reranked, 1):  
        doc = corpus[h['corpus_id']]  
        context_parts.append(f"[{i}] {doc}")  
        print(f"    [{i}] CE={h['ce_score']:+.2f} → {doc[:65]}...")  
  
    # STEP 3 – Build prompt  
    # The context is injected directly into the prompt so the LLM  
    # reads it before generating an answer. This "grounds" the answer.  
    context = "\n".join(context_parts)  
    prompt = f"""\nAnswer the following question using only the information in the co  
If the context does not contain the answer, say "I don't know."
```

Context:

```
{context}
```

Question: {question}

Answer: ""

```
  
    # STEP 4 – Generate  
    output = generator(prompt, max_new_tokens=120, do_sample=False)  
    answer = output[0]['generated_text']  
  
    print(f"\n💬 Answer: {answer}")  
    return answer
```

```
rag_answer("How much money did Interstellar make worldwide?")
rag_answer("Who was the scientific consultant for the film?")
rag_answer("What awards did Interstellar win?")
```

OUTPUT

Question: How much money did Interstellar make worldwide?

 Retrieved context passages:

[1] CE=+9.12 → The film had a worldwide gross over \$677 million...

[2] CE=+3.40 → It was nominated for five awards at the 87th Academy...

[3] CE=+1.05 → Interstellar is a 2014 epic science fiction film...

 Answer: Over \$677 million worldwide, or \$773 million including subsequent re-releases.

7.2 Advanced RAG Patterns

Simple RAG — one query, one retrieval, one answer — fails in many realistic scenarios. Here are the most important extensions.

① Query Rewriting

Problem: Real user messages are often verbose, ambiguous, or reference earlier conversation turns. The retriever gets confused by irrelevant words.

ORIGINAL USER MESSAGE

"We have an essay due tomorrow. We have to write about some animal. I love penguins. I could write about them. But I could also write about dolphins. Are they animals? Maybe. Let's do dolphins. **Where do they live for example?**"



REWRITTEN QUERY (LLM OUTPUT)

"Where do dolphins live?"

The LLM identifies the actual information need buried in a verbose message, then runs retrieval on the clean query. Without rewriting, the bi-encoder would embed noise words like "essay", "penguins", "tomorrow" and retrieve irrelevant passages.

Query rewriting via LLM prompt

PYTHON

```
# Step 1: Ask LLM to rewrite the query
rewrite_prompt = """Extract the core search query from this user message.
Output only the search query, nothing else.
```



```
User message: "{user_message}"
Search query: ""
```

Step 2: Feed the rewritten query to retrieval

```
def rag_with_rewrite(user_message):
    rewritten = generator(rewrite_prompt.format(user_message=user_message),
                          max_new_tokens=30)[0]['generated_text'].strip()
    print(f"Rewritten query: '{rewritten}'")
    return rag_answer(rewritten) # use our existing RAG function

rag_with_rewrite("We need to write about dolphins. Where do they live?")
# Rewritten query: 'dolphin habitat'
```

② Multi-Query RAG

Problem: A single question may require facts from multiple different documents. One retrieval query can't capture all of them.

ORIGINAL QUESTION

"Compare the financial results of Nvidia in 2020 vs. 2023"

SUB-QUERY 1

"Nvidia financial results revenue 2020"

SUB-QUERY 2

"Nvidia financial results revenue 2023"

↓ retrieve independently, merge and deduplicate results ↓

Combined context: 2020 revenue docs + 2023 revenue docs → LLM generates comparison

Multi-Query RAG implementation

PYTHON

```
def multi_query_rag(question: str):
    # Step 1: Generate sub-queries
    subquery_prompt = f"""Break this question into 2-3 specific search queries.
    Output one query per line, nothing else.
    Question: {question}
    Queries: """
    raw = generator(subquery_prompt, max_new_tokens=60)[0]['generated_text']
    sub_queries = [q.strip() for q in raw.strip().split('\n') if q.strip()]
    print(f"Sub-queries: {sub_queries}")

    # Step 2: Retrieve for each sub-query, merge results
    all_hits = {}
    for sq in sub_queries:
        q_emb = bi_encoder.encode(sq, convert_to_tensor=True)
```

```

hits = util.semantic_search(q_emb, corpus_embeddings, top_k=5)[0]
for h in hits:
    cid = h['corpus_id']
    # Keep the highest score if the same doc appears multiple times
    if cid not in all_hits or h['score'] > all_hits[cid]:
        all_hits[cid] = h['score']

# Step 3: Rerank the merged pool
candidate_ids = sorted(all_hits, key=all_hits.get, reverse=True)[:10]
context = "\n".join([corpus[i] for i in candidate_ids[:4]])

# Step 4: Generate answer with merged context
prompt = f"Context:\n{context}\n\nQuestion: {question}\nAnswer:"
return generator(prompt, max_new_tokens=150)[0]['generated_text']

```

③ Multi-Hop RAG

Problem: Some questions require *sequential* reasoning where the answer to Step 1 determines what to ask in Step 2. You can't formulate all queries upfront.

ORIGINAL QUESTION

"Who are the top 3 car manufacturers in 2023? Do each of them make electric vehicles?"

HOP 1

Query: "largest car manufacturers 2023 by sales"

Retrieved: "Toyota: 10.5M, VW: 9.2M, Hyundai: 7.4M units in 2023"

↓ read result → now we know who the top 3 are

HOP 2

Query 2a: "Toyota electric vehicles EV lineup" → bZ4X, bZ3, Prius PHEV

Query 2b: "Volkswagen electric vehicles EV" → ID.4, ID.3, ID. Buzz

Query 2c: "Hyundai electric vehicles EV" → IONIQ 5, IONIQ 6, Kona EV

↓ all context combined → LLM generates final answer

ANSWER

"The top 3 manufacturers in 2023 were Toyota, VW, and Hyundai. All three offer EVs: Toyota sells the bZ4X, VW offers the ID.4 and ID.3, and Hyundai has the IONIQ 5 and IONIQ 6."



Multi-Hop requires capable models

The LLM must be able to: (1) read partial results, (2) identify what's still missing, (3) formulate the next query autonomously. Small models (Flan-T5) struggle. Use Phi-3-mini, Llama-3, or GPT-4 class models for reliable multi-hop behavior.

PATTERN	PROBLEM IT SOLVES	KEY REQUIREMENT
Query Rewriting	Verbose / conversational queries	LLM prompt to extract intent
Multi-Query RAG	Question needs facts from multiple docs	Know sub-queries upfront; merge results
Multi-Hop RAG	Next query depends on previous answer	Capable LLM; iterative loop
Hybrid Search	Exact phrases + semantic queries	BM25 + bi-encoder, RRF fusion
HyDE	Query and document styles differ a lot	Generate hypothetical answer, embed it

08 Evaluation Metrics

How do we know if our retrieval system is actually good? Worked examples with hand calculations.

To evaluate a search system you need three things: (1) a **corpus** of documents, (2) a set of **test queries**, and (3) **relevance judgments** — human annotations saying which documents are relevant for each query.

8.1 MAP — Mean Average Precision

MAP rewards systems that rank relevant documents *higher up*. It is sensitive to position: a relevant doc at rank 1 is worth much more than one at rank 10.

$$\text{Precision@k} = (\# \text{ relevant docs in top } k \text{ results}) / k$$

Fraction of the top-k results that are relevant. We compute this only at positions where a relevant document appears.

$$\text{Average Precision (AP)} = (1 / R) \times \sum \text{Precision@k} \quad [\text{only at relevant positions } k]$$

R = total number of relevant documents for this query. We sum Precision@k only at positions where the returned doc is relevant, then divide by R.

$$\text{MAP} = (1 / Q) \times \sum \text{AP}(q) \quad [\text{averaged over all } Q \text{ test queries}]$$

The "Mean" in MAP = average of AP values over all queries. This gives a single number to compare two systems.

SCENARIO A – ONE RELEVANT DOCUMENT, RETURNED AT RANK 1 (PERFECT)

QUERY: "WHO DIRECTED INTERSTELLAR?" – 1 RELEVANT DOCUMENT EXISTS IN CORPUS

Rank		RELEVANT: "Directed by Christopher Nolan."	$P@1 = 1/1 = 1.000$
1			

Rank		Not relevant	skip
2			

Rank		Not relevant	skip
3			

$\text{AP} = (1/1) \times (1.000) = 1.000$ – Perfect score. Relevant doc at rank 1.

SCENARIO B – ONE RELEVANT DOCUMENT, RETURNED AT RANK 3 (PENALIZED)

SAME QUERY – BUT THE SYSTEM RETURNS TWO IRRELEVANT DOCS BEFORE THE RELEVANT ONE

Rank		Not relevant	skip
1			

Rank		Not relevant	skip
2			

Rank		RELEVANT: "Directed by Christopher Nolan."	$P@3 = 1/3 = 0.333$
3			

$\text{AP} = (1/1) \times (0.333) = 0.333$ – Position penalty: same doc, much lower score.



Why position matters: Scenarios A vs B compared

Both systems returned the single relevant document. But System A (rank 1) gets AP=1.0, System B (rank 3) gets AP=0.333. MAP captures this: **a user who stops reading after rank 2 finds nothing with System B.** The metric rewards systems that help users find answers faster.

SCENARIO C – TWO RELEVANT DOCUMENTS (MATCHING THE PDF EXAMPLE)

QUERY: "INTERSTELLAR SCIENCE ACCURACY" – 2 RELEVANT DOCUMENTS EXIST (R=2)

Rank 1  **RELEVANT: "Praised by astronomers for scientific accuracy."** $P@1 = 1/1 = 1.000$

Rank 2  Not relevant: "Premiered Oct 26, 2014." skip

Rank 3  **RELEVANT: "Kip Thorne, Nobel laureate, was scientific consultant."** $P@3 = 2/3 = 0.667$

Rank 4  Not relevant skip

Rank 5  Not relevant skip

$AP = (1/2) \times (1.000 + 0.667) = (1/2) \times 1.667 = 0.833$
Sum only at ranks 1 and 3 (where  appeared). Divide by R=2.

SCENARIO D – SAME TWO RELEVANT DOCUMENTS, BUT SYSTEM 2 RETURNED THEM IN WORSE ORDER

SYSTEM 1 – AP = 0.833 (FROM ABOVE)

Rank 1  Relevant $P@1=1.000$

Rank 2  — skip

SYSTEM 2 – RELEVANT DOCS AT RANKS 2 AND 3

Rank 1  — skip

Rank 2  Relevant $P@2=0.500$

Rank  Relevant $P@3=0.667$
3

$$AP = (1.000 + 0.667)/2 = 0.833$$

Rank  Relevant $P@3=0.667$
3

$$AP = (0.500 + 0.667)/2 = 0.583$$

FULL MAP CALCULATION – THREE QUERIES COMBINED

Now we bring together AP scores from multiple queries to compute MAP. This mirrors the example in the textbook (Figure 8-23) but with full calculations shown.

TEST SUITE: 3 QUERIES, COMPUTING MAP FOR SYSTEM 1

QUERY 1: "Interstellar science accuracy" — R=2 relevant docs

Rank  Relevant $P@1 = 1/1 = 1.000$
1

Rank  — skip
2

Rank  Relevant $P@3 = 2/3 = 0.667$
3

$$AP(Q1) = (1/2) \times (1.000 + 0.667) = 0.833$$

QUERY 2: "Who directed Interstellar?" — R=1 relevant doc

Rank  Relevant $P@1 = 1/1 = 1.000$
1

Rank  — skip
2

Rank  — skip
3

$$AP(Q2) = (1/1) \times (1.000) = 1.000$$

QUERY 3: "Interstellar box office" — R=1 relevant doc

Rank  — skip
1

Rank  —
2

skip

Rank  Relevant
3

$P@3 = 1/3 = 0.333$

$$AP(Q3) = (1/1) \times (0.333) = 0.333$$

$$MAP = (1/3) \times (AP(Q1) + AP(Q2) + AP(Q3))$$

$$MAP = (1/3) \times (0.833 + 1.000 + 0.333)$$

$$MAP = (1/3) \times 2.166 = \mathbf{0.722}$$

USING MAP TO COMPARE TWO SYSTEMS

MAP's purpose is to give a single number to decide which system is better. Here System 1 vs System 2 on the same 3 queries:

QUERY	R (# RELEVANT)	SYSTEM 1 AP	SYSTEM 2 AP
Q1: science accuracy	2	0.833	0.583
Q2: who directed	1	1.000	1.000
Q3: box office	1	0.333	0.000
MAP		0.722 ✓ better	0.528



Why MAP is called "Mean Average Precision"

Precision@k = precision at a specific rank. **Average Precision** = average of precision values at all relevant ranks, for one query. **Mean Average Precision** = mean of those AP values across all queries. The double "average/mean" is intentional — two levels of averaging.

8.2 nDCG — Normalized Discounted Cumulative Gain

MAP treats relevance as binary (relevant / not relevant). But in reality, one document can be *more* relevant than another — a perfect answer vs. a partial answer. **nDCG** handles graded relevance.

$$DCG@k = \sum_{i=1}^k \text{relevance}(i) / \log_2(i + 1)$$

The denominator $\log_2(i+1)$ is the "discount" — results at lower ranks contribute less. $\text{relevance}(i) \in \{0, 1, 2, 3\}$ for not relevant / partially / relevant / very relevant.

$$\text{nDCG@k} = \text{DCG@k} / \text{IDCG@k}$$

IDCG = Ideal DCG — the DCG you'd get if all relevant docs were ranked perfectly at the top. nDCG is always in $[0, 1]$; 1.0 = perfect ranking.

WORKED EXAMPLE — NDCG@3

GRADED RELEVANCE LABELS: 3 = HIGHLY RELEVANT, 1 = SOMEWHAT RELEVANT, 0 = NOT RELEVANT

Rank		
1	rel=3 Very relevant doc	$3 / \log_2(2) = 3.000$

Rank		
2	rel=0 Irrelevant doc	$0 / \log_2(3) = 0.000$

Rank		
3	rel=1 Somewhat relevant doc	$1 / \log_2(4) = 0.500$

$$\begin{aligned} \text{DCG@3} &= 3.000 + 0.000 + 0.500 = 3.500 & \cdot & \quad \text{IDCG@3 (ideal: ranks } 1, 2, 3 = \text{rel } 3, 1, 0) = 3.000 + 0.631 + 0 = 3.631} \\ \text{nDCG@3} &= 3.500 / 3.631 = 0.964 \end{aligned}$$

8.3 Hit@k — Did We Find Anything Useful?

Hit@k is the simplest retrieval metric. It answers one binary question: **"Is at least one relevant document in the top-k results?"** — yes (1) or no (0). No partial credit, no ranking sensitivity.

$$\text{Hit@k}(\text{query}) = 1 \text{ if any relevant doc appears in top-k results,} \\ \text{else } 0$$

Per-query it's binary. Averaged over all queries it becomes a fraction between 0 and 1.

$$\text{Hit@k} = \frac{(\# \text{ queries where at least one relevant doc is in top-k})}{Q}$$

Q = total number of test queries.

WORKED EXAMPLE – SAME 3 QUERIES, K=1 VS K=3

THREE QUERIES, SYSTEM RETURNS TOP-3 RESULTS EACH TIME

QUERY 1: "Who directed?"

Rank 1  Relevant

Rank 2 

Rank 3 

Hit@1 = 1

Hit@3 = 1

QUERY 2: "Box office?"

Rank 1 

Rank 2 

Rank 3  Relevant

Hit@1 = 0

Hit@3 = 1

QUERY 3: "Premiere date?"

Rank 1 

Rank 2 

Rank 3 

Hit@1 = 0

Hit@3 = 0

Hit@1 = (1 + 0 + 0) / 3 = 0.333 · Hit@3 = (1 + 1 + 0) / 3 = 0.667

Increasing k from 1 to 3 doubled the Hit rate – but Query 3's relevant doc was simply not retrieved at all.



Why Hit@k is Critical for RAG

In a RAG pipeline, if the relevant document is not in the retrieved top-k, **the LLM has no chance of producing a correct answer** — it has literally never seen the fact. Hit@k directly measures this: "How often does our retriever give the LLM the information it needs?" A Hit@5 = 0.80 means 20% of queries will fail regardless of how good the LLM is. **Hit@k is the hard ceiling on RAG answer accuracy.**

8.4 MRR — Mean Reciprocal Rank

MRR focuses on **where the first relevant document appears**. Unlike MAP which scores all relevant documents, MRR only cares about the single highest-ranked relevant result. It is particularly suited to tasks where the user needs exactly one correct answer.

Reciprocal Rank (RR) = 1 / rank_of_first_relevant_document

If the first relevant doc is at rank 1 → RR=1.0. At rank 2 → RR=0.5. At rank 5 → RR=0.2. If no relevant doc is found → RR=0.

MRR = (1 / Q) × Σ RR(q)

Q = number of queries. Average the Reciprocal Rank over all queries.

WORKED EXAMPLE – FOUR QUERIES

SYSTEM RETURNS TOP-5 RESULTS PER QUERY. WHERE DOES THE FIRST RELEVANT DOC APPEAR?

Q1: "Who directed Interstellar?"

Rank 1  "Directed by Christopher Nolan." ←
first hit
Rank 2  Not relevant

$$RR(Q1) = 1/1 = 1.000$$

Q2: "Scientific consultant?"

Rank 1  Not relevant
Rank 2  "Kip Thorne... scientific consultant."
← **first hit**
Rank 3 

$$RR(Q2) = 1/2 = 0.500$$

Q3: "Box office gross?"

Rank 1 
Rank 2 
Rank 3 
Rank 4  "Worldwide gross \$677 million." ←
first hit

$$RR(Q3) = 1/4 = 0.250$$

Q4: "Premiere location?"

Rank 1 
Rank 2 
Rank 3-5    — not found

$$RR(Q4) = 0 \text{ (no relevant doc found)} \\ = 0.000$$

$$\begin{aligned} MRR &= (1/4) \times (RR(Q1) + RR(Q2) + RR(Q3) + RR(Q4)) \\ MRR &= (1/4) \times (1.000 + 0.500 + 0.250 + 0.000) \\ MRR &= (1/4) \times 1.750 = 0.4375 \end{aligned}$$



MRR vs MAP — Key Difference

MRR only looks at the **first** relevant result and ignores everything below. MAP looks at **all** relevant results and their positions. If a query has 5 relevant documents all scattered across ranks 10–50, MRR might still give a good score (if rank 10 hit early enough), but MAP would penalize the system heavily for missing the other 4. Choose MRR when users need exactly one answer; MAP when completeness matters.

8.5 Which Metric to Use — Full Comparison

Each metric captures a different aspect of retrieval quality. In practice you often report several together.

METRIC	WHAT IT MEASURES	RELEVANCE MODEL	POSITION SENSITIVE?	BEST FOR	STANDARD BENCHMARK
Hit@k	Was any relevant doc in top-k? (binary per query)	Binary (0/1)	No — only cares if found, not where	RAG pipelines — sets the ceiling on LLM answer accuracy	Most RAG papers (Hit@1, Hit@5)
MRR	Rank of the FIRST relevant result	Binary (0/1)	Yes — earlier = better	QA tasks, search engines where one correct answer exists	MS MARCO (MRR@10)
MAP	Average precision across ALL relevant results	Binary (0/1)	Yes — all positions matter	Tasks with multiple relevant docs, document retrieval	TREC, older IR benchmarks
nDCG@k	Ranking quality with graded relevance	Graded (0/1/2/3)	Yes — logarithmic discount	When docs differ in relevance degree; general search quality	BEIR, MTEB (nDCG@10)
Precision@k	Fraction of top-k that are relevant	Binary (0/1)	Weak — only at cutoff k	Quick sanity check, user-facing result quality	Often reported alongside nDCG

METRIC	WHAT IT MEASURES	RELEVANCE MODEL	POSITION SENSITIVE?	BEST FOR	STANDARD BENCHMARK
Recall@k	Fraction of ALL relevant docs found in top-k	Binary (0/1)	No — only coverage matters	Measuring how much the retriever misses	Used in RAG context recall

PRACTICAL RECOMMENDATION BY USE CASE

RAG / QA SYSTEM

- ▶ Primary: **Hit@5** or **Hit@10** — does retriever give LLM what it needs?
- ▶ Secondary: **MRR@10** — how quickly does the right context appear?
- ▶ RAG quality: **Faithfulness + Answer Relevance** (Ragas)

SEARCH ENGINE / BI-ENCODER EVALUATION

- ▶ Primary: **nDCG@10** — BEIR and MTEB use this as the standard
- ▶ Secondary: **MAP@10** — if multiple relevant docs matter
- ▶ For fine-tuning: use `InformationRetrievalEvaluator`

RERANKER EVALUATION

- ▶ Measure **nDCG@10** *before* and *after* reranking
- ▶ Also report **MRR@10** — reranker should push the best result to rank 1
- ▶ Real-world gain: MIRACL bi-encoder alone 36.5 → +reranker 62.8

TURKISH NLP / DOMAIN-SPECIFIC

- ▶ Build your own test suite: 50–200 queries with relevance labels
- ▶ Report **nDCG@10** and **Hit@5** before/after fine-tuning
- ▶ Compare: `multilingual-e5-base` vs fine-tuned domain model

Computing Hit@k and MRR alongside nDCG — SentenceTransformers

PYTHON

```
from sentence_transformers.evaluation import InformationRetrievalEvaluator
from sentence_transformers import util
```

```
# InformationRetrievalEvaluator computes ALL metrics simultaneously:
# nDCG@k, MRR@k, MAP@k, Precision@k, Recall@k, Hit@k (as accuracy)
evaluator = InformationRetrievalEvaluator(
    queries=queries,          # {query_id: query_text}
    corpus=corpus,           # {doc_id: doc_text}
    relevant_docs=relevant_docs, # {query_id: set(relevant_doc_ids)}
    score_functions={"cos_sim": util.cos_sim},
```

```

    ndcg_at_k=[1, 3, 10],
    mrr_at_k=[1, 3, 10],
    map_at_k=[10],
    accuracy_at_k=[1, 3, 5, 10], # this is Hit@k
    precision_recall_at_k=[5, 10],
)

results = evaluator(model)

# Example output keys:
# cos_sim_ndcg@10      → nDCG@10
# cos_sim_mrr@10       → MRR@10
# cos_sim_map@10       → MAP@10
# cos_sim_accuracy@5   → Hit@5
# cos_sim_precision@5  → Precision@5
# cos_sim_recall@5     → Recall@5

print(f"nDCG@10 : {results['cos_sim_ndcg@10']:.4f}")
print(f"MRR@10  : {results['cos_sim_mrr@10']:.4f}")
print(f"Hit@5    : {results['cos_sim_accuracy@5']:.4f}")
print(f"MAP@10   : {results['cos_sim_map@10']:.4f}")

```

8.6 RAG Evaluation — Ragas

Retrieval metrics (MAP, nDCG) only measure whether *relevant documents were found*. For full RAG systems we also need to measure the *quality of the generated answer*. The `ragas` library automates this using an LLM-as-a-judge approach.



Faithfulness

Does the generated answer contain only information that can be found in the retrieved context? An unfaithful answer introduces facts not present in the context — a hallucination. Score: 0–1, where 1 = fully faithful.



Answer Relevance

Does the generated answer actually address the user's question? A faithful but off-topic answer still fails. Computed by asking an LLM to generate candidate questions from the answer, then checking if they match the original question.



Context Precision

Of the retrieved context chunks, how many were actually useful for generating the answer? Retrieval recall is important, but retrieving irrelevant context can confuse the LLM ("lost in the middle" phenomenon).



Context Recall

Were all the facts needed to answer the question actually retrieved? Computed against a ground-truth answer: what fraction of the ground-truth claims are supported by the retrieved context?

Using `SentenceTransformers InformationRetrievalEvaluator`

PYTHON

```
from sentence_transformers.evaluation import InformationRetrievalEvaluator
from sentence_transformers import util

# queries: dict mapping query_id → query text
# corpus: dict mapping doc_id → doc text
# relevant_docs: dict mapping query_id → set of relevant doc_ids

queries = {
    "q1": "When was Interstellar released?",
    "q2": "How much did Interstellar gross?",
    "q3": "Who was the scientific consultant?",
}
corpus = {
    "d1": "Premiered Oct 26, 2014 in Los Angeles.",
    "d2": "Worldwide gross over $677 million.",
    "d3": "Kip Thorne, Nobel laureate, was scientific consultant.",
    "d4": "Stars Matthew McConaughey and Anne Hathaway.",
}
relevant_docs = {
    "q1": {"d1"},
    "q2": {"d2"},
    "q3": {"d3"},
}

evaluator = InformationRetrievalEvaluator(
    queries=queries,
    corpus=corpus,
    relevant_docs=relevant_docs,
    score_functions={"cos_sim": util.cos_sim},
    ndcg_at_k=[1, 3, 10],
    mrr_at_k=[3, 10],
    map_at_k=[10],
)

# Returns a dict with nDCG@1, nDCG@3, MRR@3, MAP@10, etc.
results = evaluator(model, output_path="./eval_results")
print(results)
```

Summary & Quick-Start Model Reference



Key Takeaways

Semantic search = embeddings + nearest-neighbor search. A bi-encoder encodes query and document separately (independently) into a shared vector space, then retrieves the closest vectors. A cross-encoder concatenates both into one sequence and produces a single relevance score with full cross-attention — much more accurate but far too slow for first-stage retrieval. Combine them in a two-stage pipeline. Add an LLM at the end for RAG. Evaluate with MAP, nDCG, and Ragas.

Recommended Models

TASK	MODEL	NOTES
Bi-Encoder (English)	BAAI/bge-small-en-v1.5	Small, fast, top MTEB. Good default.
Bi-Encoder (English, high quality)	multi-qa-mpnet-base-dot-v1	Trained on Q&A pairs. Better for question → passage retrieval.
Bi-Encoder (Multilingual)	intfloat/multilingual-e5-base	Supports Turkish and 100+ languages.
Cross-Encoder (fast)	cross-encoder/ms-marco-MiniLM-L-6-v2	~5ms per pair. Very good quality/speed tradeoff.
Cross-Encoder (quality)	cross-encoder/ms-marco-electra-base	Slower, higher accuracy.
Multilingual Reranker	BAAI/bge-reranker-v2-m3	Best multilingual cross-encoder available.
Generator (lightweight)	google/flan-t5-large	Good instruction-following for RAG on CPU.

TASK	MODEL	NOTES
Generator (high quality)	microsoft/Phi-3-mini-4k-instruct	3.8B params. Excellent quality on GPU.